Objectifs de la feuille

- Comprendre l'architecture MVT
- Mise en place d'un template
- Exemple de vue avec un peu de style
- Héritage de templates
- URLs dynamiques

Introduction

La structure d'un projet est une des clés qui garantit sa bonne évolution, exactement comme dans votre ordinateur ! Imaginez que vous repreniez le projet d'une autre personne, ou que vous ouvriez le vôtre dans six mois. Vous ne voulez pas perdre de temps à chercher comment il est organisé ! Vous voulez aller droit au but et apporter votre modification sans pour autant tout casser.
C'est pourquoi la plupart des frameworks pensés pour le web choisissent de séparer les fichiers dans plusieurs dossiers distincts. Cela peut paraître un peu contraignant au début mais ce sont de bonnes pratiques qui garantissent la bonne évolution de votre projet.
Flask, comme vous le savez désormais, n'impose pas vraiment de structure, il la propose. S'inspirant du modèle MVC (Modèle / Vue / Contrôleur) de bien des frameworks, il se base sur trois piliers fortement imbriqués : le modèle, la vue et le template.

Mise en place d'un template

Un template est un fichier HTML qui peut recevoir des objets Python et qui est lié à une vue (nous y reviendrons). Notre vue retourne directement un string représentant un document HTML. Souvent le contenu d'une page est dynamique, et pour cela il est pratique d'utiliser des templates. Flask s'attend à trouver les templates dans le sous-répertoire `templates` de `monApp`.
Concrètement, un template peut interpréter des variables et les afficher. Par exemple, nous pouvons "donner" la variable name au template `index.html` et ce dernier affichera à la place le prénom.
Le template n'a accès qu'aux variables que la vue lui transmet, et pas aux autres. Par exemple, le template n'a pas accès à la variable db.

Créez le template `monApp/templates/index.html` avec le contenu suivant :

```
<!doctype html>
<html>
    <head>
        <title>{{ title }}</title>
    </head>
    <body>
        <h1> Bienvenu {{name}} !!</h1>
    </body>
</html>
```

Ce template est paramétré par 2 variables `title` et `name`.

Exemple d'une vue

Les vues contenues dans le fichier views.py jouent un rôle primordial : elles décident du contenu à afficher sur une page. Plus spécifiquement, ce sont elles qui génèrent le contenu à renvoyer aux requêtes qui leur sont adressées.
Une vue est une fonction qui renvoie une réponse à une requête HTTP. Toute fonction décorée par @app.route est une vue. On appelle route l'URL à laquelle va répondre la vue (par ex : /index.)

Afin de bien comprendre le rôle majeur des vues, intéressons-nous au cheminement d'une requête.

- L'utilisateur tape http://cdal.com dans son navigateur puis appuie sur entrée. Cela génère une requête HTTP de cette forme :
  GET/HTTP/1.1
  Host: cdal.com
- Le serveur à l'adresse cdal.com reçoit la requête HTTP. Il cherche dans les vues la route correspondant à la requête en fonction de la méthode HTTP utilisée et de l'URL.
- Le serveur exécute la vue : éventuellement, si la route contient des paramètres, ils sont passés comme arguments à la vue. Si une modification d'un objet en base de données est demandée, la vue fait appel au modèle. Si un template est demandé, la vue l'appelle.
- Le serveur transmet la réponse HTTP de la vue au navigateur de l'utilisateur (appelé communément client).

Le protocole HTTP est très intéressant ! Tout le web que nous connaissons l'utilise.

Afin de mettre en pratique ce que nous venons d'explorer, affichons le contenu du template index.html lorsque l'utilisateur arrive sur la page d'accueil. Pour cela, il faut modifier la vue index( ) dans le fichier views.py :

```
from .app import app
from flask import render_template

@app.route('/')
@app.route('/index/')
def index():
    return render_template("index.html",title ="R3.01 Dev Web avec Flask",name="Cricri")
```

Vous voyez ci-dessus que nous pouvons associer une vue à plusieurs URL ! Il vous suffit de les indiquer les unes à la suite des autres. Pourquoi ajouter un slash à la fin de l'URL ? Les utilisateurs peuvent taper deux adresses différentes /index ou /index/. Hé oui, ce ne sont pas les mêmes ! Si vous déclarez une route sans le slash final (/index), le serveur renverra une erreur 404 (page non trouvée) si l'utilisateur tape /index/. Mais si vous déclarez une route avec le slash final (/index/) et que l'utilisateur tape /index, le serveur ne renverra pas d'erreur. Pensez-y ! Lancez votre serveur et vérifiez le bon fonctionnement des urls :

- http://127.0.0.1:5000/index
- http://127.0.0.1:5000/
- http://127.0.0.1:5000/index/

Un peu de style

Pour ajouter du CSS, nous allons créer un fichier style.css dont le contenu est statique plutôt que dynamique. Flask s'attend à trouver les fichiers statiques dans monApp/static. Dans le fichier monApp/static/css/style.css nous plaçons le code suivant :

```
h1 {
        color : red;
}
```

Pour l'utiliser, Flask offre, par exemple, une méthode plutôt intéressante pour faire appel à des fichiers qui se trouvent dans le dossier static : url_for( ). Cette méthode est disponible dans le template. Que fait-elle ? Elle génère une URL en fonction des paramètres transmis. Le premier paramètre est le nom du dossier qui contient les fichiers que l'on souhaite lier. Le second, optionnel, est le nom d'un fichier. Par exemple :

```
url_for ('static', filename ='css/style.css')
```

Ceci générera le lien suivant : /static/css/style.css.
Intégrez-la dans le <head> du template index.html ! Mais si vous vous contentez d'insérer la méthode telle quelle dans le template, elle sera interprétée comme du HTML pur. Afin d'indiquer à Flask que vous insérez du Python, qui doit donc être interprété comme tel, entourez le code d'accolades. Comme ceci :

```
<link rel="stylesheet" href="{{ url_for ('static', filename ='css/style.css') }}">
```

Vérifiez que çela marche comme avant et que vous avez du style !

C'est à vous de jouer !

Faites en sorte que vos deux vues restantes about( ) et contact( ) rendent chacune un template about.html et contact.html, tous deux s'appuyant sur la feuille de style existante.
Vous avez le choix de la structure HTML de vos templates et du style que vous voulez appliquer sur les éléments des nouvelles pages HTML.
Modifiez vos tests unitaires sur les vues index, about et contact afin de maintenir votre taux de couverture de test !

Moteur de templates

Flask utilise le moteur de templates Jinja2 par défaut. Ce dernier nous permet d'utiliser certaines méthodes dans les templates pour nous simplifier la vie : lien vers d'autres pages, import d'autres templates... Nous allons voir tout cela en détail dans la suite de ce TP.

Nous avons réussi à afficher le template index.html et utiliser une feuille de style. Mais nous pourrions faire mieux ! Le template n'est pas qu'un fichier HTML classique car vous avez accès à de nombreux objets qui vous faciliteront la vie.

Vous avez déjà utilisé la méthode url_for( ) qui fait partie des objets disponibles dans le template pour la construction d'urls. Aussi, vous savez que config est un dictionnaire comprenant la configuration de l'application (dans notre cas, les éléments qui se trouvent dans config.py). Voici les autres :

- request : permet de connaître les détails de la requête HTTP reçue.
- session : la session actuelle. Vous pouvez en effet stocker des valeurs dans la session pour la retrouver ultérieurement. Il s'agit d'une pratique très courante pour authentifier un utilisateur et se souvenir de son passage.
- g : variables globales.
- get_flashed_messages( ) : permet de récupérer les messages flash stockés dans la session, souvent utilisés pour afficher des messages d'information (succès, erreur, etc.) à l'utilisateur après une redirection.

Nous y reviendrons plus tard.

Héritage de templates

A l'intérieur d'un même site, il est préférable que toutes les pages utilisent la même charte de présentation. Vous remarquez aisément qu'une partie du code se répète dans nos templates. Pour ce faire, on utilise un template de base qui contient le squelette de la charte de présentation, puis on spécialise ce template pour chaque type de pages du site.

Considérons notre template index.html ; Nous allons le scinder en 2 parties :

- base.html contenant le template de base :

```
<html>
        <head>
                <title>{{title}}</title>
                <link rel="stylesheet" href="{{ url_for ('static', filename ='css/style.css') }}">
        </head>
        {% block main %} {% endblock  %}
</html>
```

- et index.html qui spécialise ce template de base pour notre page index.

```
{% extends "base.html" %}

{% block main %}
<body>
        <h1>Bienvenu {{name}} !!</h1>
</body>
{% endblock %}
```

Vérifiez que cela fonctionne comme avant. Qu'avons nous fait ? Jinja2 nous permet de créer un template que l'on pourra étendre à volonté, comme une classe enfant qui hériterait d'une classe parent : vous pouvez changer les valeurs par défaut du template parent. C'est assez simple !

Le template base.html contient le squelette de toutes nos pages. Dans notre cas, la section head mais on pourrait y rajouter un menu de navigation, un footer, etc. Nous y reviendrons plus tard. Le template index.html contient alors ce qui sera amené à changer dans le template d'origine : le contenu du body.

Nous indiquons que nous souhaitons étendre base.html dans le template index.html en utilisant le mot-clé extends "base.html". Le template parent agit un peu comme un texte à trou : nous indiquons les endroits qui seront amenés à changer en utilisant le mot-clé {% block nom_du_bloc %} {% endblock %}. Jinja2 sait maintenant que du contenu peut être inséré à cet endroit. Dans le template enfant, index.html, nous englobons le contenu à envoyer dans le même bloc (ici le nom du bloc est main).

Vous pouvez créer autant de blocs que vous le voulez ! Il vous suffit de changer le nom du bloc.

C'est à vous de jouer !

Je vous laisse en faire de même pour les templates about.html et contact.html.

Créez une page grâce aux URL dynamiques

Notre page index.html est bien jolie mais il est grand temps de la rendre dynamique ! Il va s'en dire que notre internaute ne s'appellera pas toujours Cricri ! Il faudrait donc adapter le message au prénom de l'internaute. Vous devez par conséquent récupérer le prénom. Il faut donc passer ce prénom en paramètre de l'url. Par exemple :

localhost:5000/index?name=Cricri

Décomposons cette url :

- localhost:5000 : nom du domaine et port utilisé.
- /index: chéma utilisé pour identifier la route.
- ? : séparateur entre le schéma et les paramètres.
- name=Cricri : association entre le nom du paramètre (name) et sa valeur (Cricri).

C'est ce que nous appelons une URL dynamique car la valeur du paramètre est amenée à varier, voire même un paramètre inexistant. Pour récupérer ce paramètre optionnel de notre URL dynamique, Flask contient l'objet request qui représente la requête exécutée par le client.

Il contient de nombreux paramètres intéressants :

- request.path renvoie la route demandée
- request.method renvoie la méthode HTTP utilisée
- request.args renvoie un dictionnaire contenant les paramètres présents dans l'URL.

Nous allons utiliser request.args dans la vue index( ) du fichier views.py :

```
from .app import app
from flask import render_template, request

@app.route('/')
@app.route('/index/')
def index():
    # si pas de paramètres
    if len(request.args)==0:
        return render_template("index.html",title="R3.01 Dev Web avec Flask",name="Cricri")
    else :
        param_name = request.args.get('name')
        return render_template("index.html",title="R3.01 Dev Web avec Flask",name=param_name)
```

Voilà pour ce qui est des paramètres optionnels. La méthode render_template( ), que nous utilisons dans la vue, est un peu spéciale. Vous pouvez lui donner en paramètres toutes les variables que vous souhaitez utiliser dans le template.

C'est à vous de jouer !

Avec cette modification de la vue index, je vous laisse remettre à niveau votre couverture de test.

En résumé :
Un projet Flask se structure à partir de trois piliers : le modèle (TP n°2) , la vue et le template (TP n°3).
Un modèle représente la structure d'un objet de la base de données.
Un template est un fichier HTML dans lequel on peut récupérer des objets Python. Une vue a la responsabilité de traiter le contenu à envoyer à l'URL indiquée par la route. Une route dans Flask se crée en utilisant @app.route.
Jinja est un moteur de template. Il permet d'écrire du code Python dans un template (en plus du HTML). Pour indiquer à Jinja qu'on utilise des objets Python, il faut utiliser les syntaxes{{ }}, {% %} ou {{# #}} selon le code écrit. Pour insérer des objets Python dans un template, il faut les passer en paramètres de la fonction render_template au niveau de la vue.
Jinja permet d'étendre des templates grâce à l'héritage. Pour éviter de dupliquer du code, vous pouvez importer des templates HTML au sein d'un autre template. Les templates de votre projet utilisent désormais Jinja. Passons maintenant à la dynamisation de vos pages.